70 Scenario-Based Python Programming Tasks

Core Topics: Lists, List Methods, Indexing, Slicing, Nested Lists, & Flow Control

---

Scenario 1: Temperature Tracker

Given a list of 7 daily temperatures, write a loop to print only the days where the temperature exceeded 30°C.

Scenario 2: User Signups

You have a list of new usernames. Loop through them and check if a username contains spaces. If it does, print an error message.

Scenario 3: Premium Inventory Check

Given an inventory list, find the first element using positive indexing and the last element using negative indexing, then check if either is 'Premium Item' using an if-statement.

Scenario 4: Sensor Reset

A stream of sensor readings contains negative numbers representing system faults. Loop through the list and print 'Fault Data' whenever a negative value is encountered.

Scenario 5: Cart Price Cap

Given a list of product prices in a shopping cart, iterate over the list and if any item costs over $500, trigger a conditional print stating 'Requires Manager Approval'.

Scenario 6: VIP Guest Validation

Given a list of registered VIP guests, use indexing to read the third guest on the list. Use a flow control condition to check if their name starts with 'Dr.'.

Scenario 7: **Odd ID Filtration**

You receive a batch of numeric user IDs. Use a loop and conditional statement to print out only the IDs that are odd numbers.

Scenario 8: **Word length Alert**

Iterate over a list of string messages. Print an alert phrase if any message string length is shorter than 5 characters.

Scenario 9: **Stock Threshold**

A list contains the current stock counts of various items. Loop over this inventory and flag any items with a count of 0 as 'Out of Stock'.

Scenario 10: **Score Range Evaluator**

Loop through a list of student exam scores. For each score, print 'Pass' if it is 50 or above, otherwise print 'Fail'.

Scenario 11: **Weekend Log Extraction**

Given a 7-day server log list, extract the last two days (weekend logs) using list slicing, and loop through them to verify if they contain any errors.

Scenario 12: **Morning Rush Analysis**

A list tracks hourly storefront foot traffic from 6:00 AM to 10:00 PM. Slice out the morning peak hours (8:00 AM to 11:00 AM) and use flow control to check if traffic exceeded 100 people at any point.

Scenario 13: **Batch Data Split**

You have a telemetry log of 10 items. Slice the first half into a new list and the second half into another. Print which batch has a higher initial value using a conditional comparison.

Scenario 14: **Reverse Engineering logs**

A system updates a list chronologically, but you need to read it backward. Slice the entire list in reverse order, then use a loop to process items until a specific keyword 'STOP' is found.

Scenario 15: **Alternate Security Scanning**

Given a list of network packet logs, slice the list to extract every alternate packet (even indices). Use a loop to check if any of these alternate packets contain unauthorized ports.

Scenario 16: **Middle Element Extraction**

For an ongoing log analysis, write a conditional statement to check if a list length is odd. If odd, slice out the single middle element; if even, slice the middle two elements.

Scenario 17: **Archiving Active Records**

You have a historical list of projects. Slice the oldest 5 projects. Loop over the slice and print their names if their completion status flag is true.

Scenario 18: **Recent Transactions Review**

Given a transaction log list, slice the most recent 3 entries. Loop through them; if any transaction value is negative (a refund), flag it for auditing.

Scenario 19: **Trimmed Noise Data**

A signal data list contains microphone noise at the exact start and end. Slice the list to remove the first and last elements, then use a loop to calculate if the remaining data contains zero values.

Scenario 20: **Sub-list Pattern Check**

Slice out elements from index 2 to 7 from a sensor tracking stream. Run a loop over this subset to determine if all values are strictly increasing.

Scenario 21: **Dynamic Cart Assembly**

Start with an empty cart list. Loop through an incoming customer order stream. Use `.append()` to add items, but if an item is 'Banned', skip it entirely using `continue`.

Scenario 22: **Queue Management**

A waiting line list updates dynamically. When a customer is served, use `.pop(0)` to remove them. If the line becomes empty, change a status variable to 'Idle'.

Scenario 23: **Duplicate Prevention Registry**

Loop over an unverified list of attendee emails. Before adding each email to a master registry list using `.append()`, check if it already exists to prevent duplicates.

Scenario 24: **Urgent Task Prioritization**

You have a daily schedule list. Loop through incoming work tickets. If a ticket is marked 'EMERGENCY', use `.insert(0, ticket)` to push it to the top.

Scenario 25: **Inventory Cleanup**

A warehouse list tracks expired and valid products. Use a loop to find all occurrences of 'EXPIRED' and use `.remove()` to clear them out safely without breaking the program.

Scenario 26: **Merge Regional Inventories**

You have two lists of store items: `east_inventory` and `west_inventory`. If `west_inventory` contains more than 10 items, use `.extend()` to merge it into `east_inventory`; otherwise, append items one by one via a loop.

Scenario 27: **Search and Flag Record**

Given a customer database list, check if a target name exists using `.count()`. If the count is greater than 0, use `.index()` to find its exact position and update its entry to 'Verified'.

Scenario 28: **Leaderboard Ordering**

A gaming application receives a new batch of player scores. Append the scores to an existing list, then use `.sort(reverse=True)` to order them. Print an announcement if the top score broke the historical record.

Scenario 29: FIFO Buffer Purge

A data stream fills a buffer list. When the buffer length surpasses 5 elements, use a `while` loop with `.pop()` to remove elements from the end until the length drops back to 3.

Scenario 30: System Reset Command

An operating system monitors running tasks. If a specific system error keyword occurs, run a flow control statement that executes `.clear()` to instantly wipe the task registry clean.

Scenario 31: Palindromic Verification Workflow

Given a sequence of characters in a list, copy the list and use `.reverse()` on the copy. Use a conditional check to see if the original and reversed lists match exactly.

Scenario 32: Frequency Threshold Alert

A network log records IP access hits. Use `.count()` to see how many times a malicious IP appears. If it appears more than 5 times, trigger an immediate block status update.

Scenario 33: Conditional Value Replacement

Iterate over a list of currency inputs. If an element equals 'USD', find its index using `.index()` and replace it with 'United States Dollar' directly in the list.

Scenario 34: Dynamic Stack Limit

Implement a simple undo-stack list. When a user performs an action, check if the stack has reached a maximum size of 10. If it has, remove the oldest action using `.pop(0)` before appending the new action.

Scenario 35: Alphabetic Warehouse Sort

A supply shipment arrives with names out of order. Apply `.sort()` to the list, then loop through the sorted items to verify if any item names violate naming conventions (e.g., containing numerical digits).

Scenario 36: **Grayscale Image Pixel Guard**

A nested list represents a 3x3 pixel grid of an image. Iterate through each row and pixel using nested loops. If any pixel value is less than 0, force it to 0 using coordinates.

Scenario 37: **Company Department Payroll**

You have a nested list where each sub-list contains the salaries of employees in a specific department. Run nested loops to calculate the total budget for each department and print an alert if any department exceeds $50,000.

Scenario 38: **Seating Chart Booking**

A movie theater seating configuration is mapped as a nested list of strings ('Available' or 'Booked'). Write a script that checks row 2, column 4 using direct indices, and if 'Available', changes it to 'Booked'.

Scenario 39: **Warehouse Matrix Check**

A 2D nested list represents warehouse shelves, where each element is an item string. Loop through the rows. If a shelf contains the item 'Hazardous Chemical', use a break statement to halt the outer loop and sound an alarm.

Scenario 40: **Student Grade Book Tracker**

A nested list stores student records: `[[Name, Math_Grade, Science_Grade], ...]`. Loop through the records. If a student's average grade is below 60, append their name to a separate 'Academic Review' list.

Scenario 41: **Tic-Tac-Toe Win Evaluator**

A 3x3 nested list holds values 'X', 'O', or ' '. Write a flow control check to evaluate if the first row has three identical marks ('X' or 'O') signifying a row win.

Scenario 42: **Game Map Obstacle Radar**

A game board is a 2D grid grid layout. Your character is at coordinates [0][0]. Use flow control to inspect neighbor cells `[0][1]` and `[1][0]`. If both contain an 'Obstacle', print 'Trapped!'.

Scenario 43: **Quarterly Financial Auditing**

A nested list contains 4 sub-lists, each representing monthly revenue for one quarter. Loop through each quarter. If any month's revenue dropped below $10,000, log that specific quarter index for review.

Scenario 44: **Multi-City Temp Warning**

A nested list contains temperature datasets for 3 cities over 5 days. Use a nested loop configuration to evaluate each temperature. If any city registers a temperature over 45°C, log a heatwave warning along with the city index.

Scenario 45: **E-commerce Category Sweep**

An e-commerce store structures its inventory as a nested list of item categories. Run a nested loop to identify any empty sub-lists. If found, populate them with a placeholder string 'Coming Soon'.

Scenario 46: **Flight Matrix Status**

A 2D grid tracks airplanes on a radar grid. Each element contains either an altitude number or `None`. Loop over the matrix; if two planes are found in adjacent columns within the same row, trigger a collision warning.

Scenario 47: **Data Cleaning Blank Remover**

A nested list contains survey feedback text entries. Loop through each sub-list. If any individual feedback item is an empty string, use `.remove()` to extract it from that specific sub-list.

Scenario 48: **Restaurant Menu Filter**

A restaurant structures its menu as a nested list: `[[Dish_Name, Price, Is_Vegetarian], ...]`. Use a loop to iterate through the menu items and print only the names of dishes that are vegetarian and cost under $15.

Scenario 49: **Delivery Matrix Costing**

A coordinate grid represents delivery zones where the cell value is the baseline shipping cost. Loop over the matrix; if a cell's shipping cost is greater than $25, apply a 10% holiday discount directly to that cell.

Scenario 50: **Nested ID Registry Match**

You have a list of user session lists: `[[Session_ID, User_ID], ...]`. Given a target User_ID, loop through the nested structure to find and collect all matching Session_IDs connected to that user.

Scenario 51: **Baking Oven Grid Monitor**

A nested list maps industrial bakery trays, tracking cookie bake levels. Loop through the grid. If a cookie's bake level reaches 10, change its state string to 'Baked'. If it exceeds 12, flag it as 'Burnt'.

Scenario 52: **Social Media Tag Extraction**

A list contains sub-lists of user-generated post hashtags. Loop through each sub-list. If a hashtag list contains more than 5 tags, use slicing to keep only the first 3 tags to prevent spamming.

Scenario 53: **Bus Route Schedule Alignment**

A nested list stores arrival times for different bus lines. Loop through the lines. If a specific line has fewer than 2 scheduled arrivals left, append a placeholder time string to that line's list to maintain system stability.

Scenario 54: **Corporate Directory Search**

A nested list profiles company teams: `[[Manager_Name, [Team_Members]], ...]`. Write a nested loop that searches for a specific employee name. When found, print the name of their corresponding manager.

Scenario 55: **Digital Asset Grid Compression**

A 2D color channel grid contains numeric values. Run nested loops to divide every number by 2 (downsampling) if the matrix row length is exactly equal to the column length.

Scenario 56: **Log Data Purge Pipeline**

A server tracking system maintains an active list of system logs. If the log count exceeds 50 entries, slice the oldest 20 entries, clear the main list, and use `.extend()` to save only the remaining fresh records.

Scenario 57: **Dynamic Grade Normalization**

A teacher keeps a nested list of raw exam scores for multiple classrooms. Loop through each classroom list. If a classroom's highest score is below 80, apply a loop to add 5 bonus points to every student in that specific classroom.

Scenario 58: **Inventory Serialization Check**

A stockroom list holds alphanumeric item serial codes. Loop through the codes. If a code starts with 'SN-', use slicing to remove the prefix, and then append the clean numeric portion to a master database registry list.

Scenario 59: **Automated Banking Audit**

A nested list tracks daily customer banking interactions: `[[Account_Number, Balance, [Recent_Transactions]], ...]`. Loop through the accounts. If an account has a negative balance, loop through its transaction history slice to trace the largest debit value.

Scenario 60: **Security Access Filter**

A master access list records keycard swipe IDs. Create a flow control script that loops through the list, filters out any IDs that fail length requirements, reverses the filtered subset, and appends them to an inspection queue.

Scenario 61: **Smart Home Schedule Sync**

A nested list maps weekly smart device states hourly. Loop through each day's sub-list. If the device detects a 'Vacation Mode' flag via an if-statement, slice all subsequent hours for that day and set them to 'Power Save'.

Scenario 62: **Student Attendance Audit**

A nested list logs daily classroom roll calls: `[['Present', 'Absent', 'Present'], ...]`. Loop through the main list. If a classroom contains more than 3 'Absent' values using `.count()`, append that entire classroom index to an administrative review list.

Scenario 63: Flight Luggage Scalper

A passenger plane luggage weight manifest is stored as a list of numbers. Use a `while` loop to continuously check the sum of the list. If it exceeds safety standards, use `.pop()` to remove the heaviest bag until weight drops below the threshold.

Scenario 64: E-commerce Wishlist Pruning

A customer profile contains a nested list of product items. Loop through the categories. If a product item's price is reduced by a sale flag, use `.insert()` to bring it to index 0 of that category list to alert the customer.

Scenario 65: Traffic Radar Data Flattening

A multi-lane highway camera generates a nested list of recorded car speeds per lane. Loop through the lanes, extract speeds over 120km/h using conditionals, and append them into a flat, single-dimensional 'Speeding Violators' list.

Scenario 66: Supply Chain Quality Assurance

A shipping crate carries an array of fruit quality scores. Slice out the middle section of the array. Loop through the slice; if any fruit item scores under a quality rating of 5, clear the entire master shipping list using `.clear()` to reject the batch.

Scenario 67: Sensor Outlier Interpolation

A continuous telemetry stream tracks system performance variables. Run a loop with index tracking. If a single value fluctuates dramatically compared to its neighboring elements (accessed via slicing or offset indexing), replace it with the average of its neighbors.

Scenario 68: Subscription Tier Upgrader

A subscription registry list contains metadata sub-lists: `[[User_id, Tier, Points], ...]`. Use a loop to scan the accounts. If points exceed 1000, change the Tier string element to 'Gold' and use `.append()` to add a 'Bonus Item' into a nested rewards inventory list inside that record.

Scenario 69: Tournament Bracking Split

An open-source e-sports tracker stores player objects sequentially in a list. Use a conditional statement to check if the total player count is a power of 2. If valid, slice the list into two perfectly equal halves ('Bracket A' and 'Bracket B') for processing.

Scenario 70: Industrial Assembly Line Error Fix

An automation conveyor belt records component logs as strings. Loop over the sequence. If a bad part code appears, use `.index()` to discover its location, apply an active correction update, and then use `.reverse()` on the past 5 elements to verify structural alignment.